NODEBASE — INTERNAL ENGINEERING DOCUMENT

Saving a Credential

End-to-end trace of the application's credential-storage feature: what the browser sends, where the API key is transformed, which tables are written, what each stored field actually contains, and how the secret is recovered at execution time.

Scope — the `credentials` feature, not account sign-in **Tables touched** — `Credential`, `Node`

Evidence — live reads from the project's Neon database

This document covers **saving an API key or service-account JSON in the app** (the `credentials` feature). Account sign-up, the OTP email flow, and session handling are documented separately in `otp-email-flow.pdf` and `login-flow.pdf`; the `Account` table covered there is a different table with a different purpose and is not the subject here.

1. The shape of the feature

Nodebase integrations are third-party services that require a key. Rather than hard-coding one per deployment, each user stores their own keys and attaches them to the nodes that need them. A credential is therefore a small, user-scoped, encrypted blob, and the interesting part of the flow is everything that happens to that blob on the way in and on the way out.

The three pages

ROUTE	COMPONENT AND PURPOSE
`/credentials`	`credentials/page.tsx` → `credentials.tsx`. Paginated, case-insensitive searchable list. Nuqs-backed params (`params.ts`, `use-credentials-params.ts`).
`/credentials/new`	`new/page.tsx` → `CredentialForm` with no `initialData`. Server prefetch of the user's subscription via `prefetch.ts`.
`/credentials/<id>`	`[credentialId]/page.tsx` → prefetches one credential, renders `CredentialView`. This is both the detail view and the edit form.

The five procedures and who may call them

All six live in `src/features/credentials/server/routers.ts` and are registered in `src/trpc/routers/_app.ts`. The gate matters: `premiumProcedure` requires an active Polar subscription and throws `FORBIDDEN` otherwise.

PROCEDURE	GATE	KIND	BEHAVIOUR
`create`	**premium**	mutation	Inserts one row. Requires a subscription.
`update`	protected	mutation	Overwrites name, type, and re-encrypts the value. *No* subscription check.
`remove`	protected	mutation	Hard delete, ownership-scoped.
`getOne`	protected	query	Single row, ownership-scoped. *Returns the encrypted value.*
`getMany`	protected	query	Paginated, `name contains` with `mode: "insensitive"`, ordered by `updatedAt desc`. Also returns the encrypted value.
`getByType`	protected	query	All of one `CredentialType`, ownership-scoped, `updatedAt desc`. Backs the picker in every node dialog.

Gate asymmetry, stated plainly
Only `create` is a `premiumProcedure`. Once a credential exists, a lapsed subscriber can still rotate the key, rename it, and delete it through `update` and `remove`, both of which are merely `protectedProcedure`. That is defensible — you should be able to revoke your own key after cancelling — but if the intent was "premium feature" rather than "premium acquisition", the gate is not where you think it is.

2. The browser side

`CredentialForm` (`credential.tsx:95`) is a `react-hook-form` bound to a Zod schema that mirrors the router input exactly — `name: z.string().min(1)`, `type: z.enum(CredentialType)`, `value: z.string().min(1)` (`credential.tsx:50-54`). Client and server validation agreeing is deliberate; the router re-validates regardless, because the client check is a convenience and not a control.

Five credential types are offered, each with its own logo asset (`credential.tsx:58-84`):

ENUM VALUE	LABEL	WHAT THE VALUE FIELD HOLDS
`ANTHROPIC`	Anthropic	Four of the five are a single API key rendered in `<input type="password">`. The fifth is different — see below.
`GEMINI`	Gemini	
`OPENAI`	OpenAI	
`TELEGRAM`	Telegram	
`GOOGLE_SHEETS`	Google Sheets	

The one type that is not an API key

Selecting `GOOGLE_SHEETS` swaps the single-line password input for a `<Textarea>` labelled "Service Account JSON" (`credential.tsx:207-227`), pre-populated with a placeholder showing the `type`, `client_email`, and `private_key` shape. An `Alert` with a four-step accordion then walks the user through enabling the Sheets API, creating a service account, downloading the JSON key, and sharing the spreadsheet with the service account's email.

This is the largest secret the system handles, and the database bears that out: the `google-sheet-1` credential is **4,988 hex characters** of ciphertext — about 2,398 bytes of plaintext, which is a realistic service-account key including its embedded `-----BEGIN PRIVATE KEY-----` block. The other three credentials are one or two orders of magnitude smaller.

> **The `private_key` in that JSON is a real RSA private key**
>
> The form's own warning ("Never commit it to a public repository", `credential.tsx:330-333`) is the right advice but it is aimed at git, not at this database. Once saved, the key is encrypted at rest and, as documented in section 5, recoverable by anyone holding `ENCRYPTION_KEY`. It is also *not* rotatable in place: changing the service account in Google Cloud produces a new key, and there is no re-encryption path that avoids re-entering it.

3. The write path

PHASE 1 — CLIENT TO SERVER

1 Submit

`onSubmit` (`credential.tsx:116`) branches on `isEdit`. Create calls `createCredential.mutateAsync(values)`; edit calls `updateCredential.mutateAsync({ id, ...values })`. The `value` travels as a plain string in the tRPC request body over HTTPS.

2 React Query mutation fires

`useCreateCredential` (`use-credentials.ts:25`) registers `trpc.credentials.create` with an `onSuccess` that raises a `sonner` toast with the returned name and invalidates `getMany` so the list refetches. `onError` routes through `useUpgradeModal` so a `FORBIDDEN` rejection surfaces as an upgrade prompt rather than a raw error.

3 Gate, then validate, then encrypt

Server-side, in order: `premiumProcedure` resolves the session and subscription; Zod re-parses the body; `prisma.credential.create` runs with `value: encrypt(value)` (`routers.ts:24-31`).

> The encrypt call is the *last* thing that happens before persistence. Nothing in the pipeline ever writes the plaintext to disk, to a log, or to a queue — contrast this with the OTP flow, where the code *is* placed in BullMQ job data.

4 Redirect

On success the client navigates to `/credentials/${data.id}` (`credential.tsx:125`). The route already has a server prefetch for that id, so the page renders already populated — no second round trip on paint.

4. Encryption at rest

`src/lib/encryption.ts` is thirteen lines and hides everything:

```
let _cryptr: Cryptr | null = null;

function getCryptr(): Cryptr {
  if (!_cryptr) {
    _cryptr = new Cryptr(process.env.ENCRYPTION_KEY!);
  }
```

```
  return _cryptr;
}

export const encrypt = (text: string) => getCryptr().encrypt(text);
export const decrypt = (text: string) => getCryptr().decrypt(text);
```

Read from the installed `cryptr` package, the construction is deterministic in its parameters and explicit in its layout:

PARAMETER	VALUE	NOTES
Algorithm	`aes-256-gcm`	Authenticated encryption. Tampering with the ciphertext fails the tag check rather than yielding garbage.
Key derivation	`PBKDF2-SHA512`	100,000 iterations, 32-byte derived key.
Salt	`64 bytes`	Fresh random salt per encryption, stored with the ciphertext.
IV	`16 bytes`	Fresh random IV per encryption.
Auth tag	`16 bytes`	From `cipher.getAuthTag()`.
Encoding	`hex`	Whole buffer concatenated, then hex-encoded.

The stored layout

Cryptr concatenates four values in a fixed order and hex-encodes the result:

```
return Buffer.concat([salt, iv, tag, encrypted]).toString("hex");
```

```
byte offset â†’
â”Œâ”€â”€â”€â
```

Two unrelated users both named a credential `google-sheet-1`, and they hold entirely different things — one a 2,398-byte service account, the other a 10-byte OpenAI key. Names are per-user labels, not global identifiers, and carry no constraint beyond non-emptiness.

> **The salt and IV live in the same column as the ciphertext**
>
> Because the salt is stored alongside the data, a row is self-describing: anyone who obtains `ENCRYPTION_KEY` can decrypt everything with no external state. The flip side is that **losing `ENCRYPTION_KEY` loses every credential permanently and irreversibly**. There is no key escrow, no rotation path, and no re-encryption utility anywhere in the repository — the only two exports are `encrypt` and `decrypt` (`encryption.ts:12-13`). A key rotation today means decrypting with the old key and re-encrypting with the new one in a script that does not yet exist.

> **Encryption is not deterministic, by design**
>
> The same API key saved twice produces two different ciphertexts, because salt and IV are random per call. Measured: **0 rows** in the database share identical `value` strings. This is correct for GCM — reusing an IV under one key would be catastrophic — but it has a product consequence: **you cannot detect that a user has already saved a particular key**, and you cannot enforce key uniqueness. Dedup would require decrypting and comparing, which is the opposite of what you want to do routinely.

One performance detail: `getKey` calls `crypto.pbkdf2Sync` with 100,000 SHA-512 iterations, synchronously, on the request thread (`cryptr/index.js`). Each encrypt or decrypt therefore blocks the Node event loop for a measurable interval. The module-level cache means the `Cryptr` instance is built once, but the PBKDF2 work is repeated on every single call — relevant if you ever batch-import credentials or decrypt in a loop. The `CryptrAsync` export in the same package is the drop-in fix.

5. Which tables are written, and what goes in them

The `Credential` table

Physical name `"Credential"` — capitalised, because unlike the four Better Auth models this one has no `@@map`. That inconsistency is worth knowing before you write raw SQL: the auth tables are lowercase (`user`, `session`, `account`, `verification`, per `@@map` at lines 31/46/65/76) while the application tables are PascalCase.

COLUMN	TYPE	CONTENT AND NOTES
`id`	`String @id`	`cuid()`, client-generated. e.g. `cmnj019160001vc8srpunymsm`. This is the value stored inside node data.
`name`	`String`	Free-text user label, non-empty. **Stored in plaintext.** Searchable with a case-insensitive `contains`, so it is indexed in practice and will appear in logs and admin tooling.
`type`	`CredentialType`	Postgres enum of five values. Drives which integrations can use it, and which SDK the executor constructs.
`value`	`String`	**The encrypted secret.** 192 fixed hex chars of overhead plus the ciphertext. Never plaintext at rest.
`userId`	`String`	Owner. Foreign key to `User.id` with `onDelete: Cascade` — deleting the user deletes every credential, with no application code required.
`createdAt / updatedAt`	`DateTime`	`now()` on create; `@updatedAt` on write. `getMany` orders by `updatedAt desc`, so a re-encrypted key jumps to the top of the list — a handy signal that a rotation happened.

The `Node` table — and the column that is never used

A credential becomes useful only once a node references it. Here is the part that does not match the schema.

The model declares a real foreign-key column:

```
model Node {
  id           String      @id @default(cuid())
  workflowId   String
  name         String
  type         NodeType
  credentialId String?                           // â†’ line 141
  credential   Credential? @relation(fields: [credentialId], references: [id])
  position     Json
  data         Json        @default("{}")
}
```

Nothing ever writes it. Measured across all **46 rows** in the `Node` table: `count("credentialId") = 0`. The node persistence path in `workflows/server/routers.ts:152-163` upserts `id`, `workflowId`, `name`, `type`, `position`, and `data` — and never mentions `credentialId`. The node dialogs do not send it either; they put it inside the `data` object.

What the database actually shows:

LIVE — THE REAL LINKAGE MECHANISM

```
=== WHERE THE LINK LIVES ===
node rows                                : 46
  ...with "credentialId" column set      : 0   // dead column
  ...with data->>'credentialId' present  : 7   // the real link

=== THE 7 LINKED NODES ===
GEMINI                 -> cmnj019160001vc8srpunymsm
GEMINI                 -> cmnj019160001vc8srpunymsm
```

```
GEMINI                -> cmnj019160001vc8srpunymsm
GEMINI                -> cmnj019160001vc8srpunymsm
TELEGRAM_SEND_BUTTONS -> cms9eclds0003vc0ws6wbp2rl
TELEGRAM_SEND_MESSAGE -> cms9eclds0003vc0ws6wbp2rl
TELEGRAM_SEND_MESSAGE -> cms9eclds0003vc0ws6wbp2rl
```

So the credential reference is a **JSON key inside an untyped column**, not a foreign key. The consequences are concrete:

CONSEQUENCE	DETAIL
No referential integrity	The database cannot stop a node referencing a credential that no longer exists, or one owned by a different user. Deleting a credential leaves dangling JSON pointers — the nodes still say `data->>'credentialId'` points at a row that is gone. The executor catches this at runtime as "Credential not found", not at write time.
No cascade on delete	Had the column been used, `Credential` would need a reverse relation to clear it. Instead nothing rewrites the affected nodes, so a deleted credential silently breaks every workflow that referenced it.
No database-level join	"Which workflows use this key?" requires `data ? 'credentialId'` — a JSONB scan — rather than an index seek on a real column.
Deleting a user is still safe	This one is fine. `Credential.userId` *is* a real foreign key with `onDelete: Cascade`, so the credentials go with the user; the workflow and node rows cascade from the user in turn. No orphaned secrets survive.

For what it is worth, storing the reference in `data` is not unreasonable — `Node.data` is already a per-type JSON config blob that the whole editor round-trips, so a credential id is at home there, and it is why `workflows.update` needs no credential awareness at all. The problem is that the schema *also* declares a column for it. Two mechanisms for one relationship, one of them dead, is a trap for the next reader. Either populate the column and drop it from `data`, or delete lines 141-142 and document the JSON convention.

Summary — every table this feature touches

TABLE	OPERATION	DATA WRITTEN	NOTES
`"Credential"`	**INSERT** (`create`)	`id` (cuid), `name` (plaintext), `type` (enum), `value` (AES-256-GCM ciphertext), `userId`, `createdAt`, `updatedAt`	Exactly one row per save. Premium-gated.
`"Credential"`	UPDATE (`update`)	All three user fields. `value` is **re-encrypted from scratch**, producing a completely different ciphertext.	Not gated by subscription. Bumps `updatedAt`.
`"Credential"`	DELETE (`remove`)	—	Hard delete. Does **not** clear `credentialId` from any node's `data`.
`"Credential"`	SELECT	—	`getOne`, `getMany`, `getByType`, and all six executors. Every one is scoped by `userId`.
`"Node"`	UPDATE (`data`)	`data.credentialId` — the credential cuid, inside the JSON blob	Written when a node dialog is saved via `workflows.update`. The `credentialId` *column* is never touched.
`"User"`	read only	—	Source of `ctx.auth.user.id`; cascade target for `Credential.userId`.
`"Session"`	read only	—	Validates the session that `premiumProcedure` / `protectedProcedure` require.

One good piece of news, checked

Because `credentialId` is a cuid the *user* supplies through their own node config, an executor must not trust it blindly. All six executors scope the lookup by owner — `prisma.credential.findUnique({ where: { id: data.credentialId, userId } })` — so pasting another user's credential id into a node yields `null` and a clean "Credential not found" error rather than someone else's API key. Verified individually in `anthropic`, `gemini`, `google-sheets`, `openai`, `telegram-send-buttons`, and `telegram-send-message`. The router enforces the same discipline on `update`, `remove`, and `getOne` by including `userId` in the `where` clause.

6. Reading the secret back at execution time

Saving is the easy half. The secret must come back out, and the rule is that it is decrypted **only** inside an executor, at the last possible moment.

PHASE 2 — WORKFLOW EXECUTION

1 Validate presence

Each executor opens by checking `data.credentialId` and throwing `NonRetriableError("… Credential is missing")` if absent. Non-retriable is the correct classification: a missing key will still be missing on the next attempt, so retrying wastes the budget.

2 **Fetch, scoped to the owner**

Inside `step.run("get-credential", …)` so the Inngest step is cached and memoised across retries (`openai/executor.ts:77-84`). The `where` clause carries `userId`, as noted above.

3 **Decrypt into the SDK constructor**

The plaintext exists for exactly one expression:

```
const openai = createOpenAI({ apiKey: decrypt(credential.value) });
```

It is never assigned to a variable that is logged, serialised, or attached to the execution record. For `GOOGLE_SHEETS` the same call yields a JSON string handed to the Sheets client (`google-sheets/executor.ts:110`); for the Telegram nodes it becomes a bot token (`telegram-send-message/executor.ts:108`).

4 **Never persisted**

The decrypted value is not written to `Execution`, to the realtime channel payload, or to the workflow context that gets threaded between nodes. A scan of all 46 nodes for secret-shaped strings — `sk-…`, `AIza…`, `-----BEGIN`, long bot tokens — returned **0 matches**. Whatever an executor produces travels as API responses, not as stored secrets.

The one place ciphertext reaches the browser

Worth stating explicitly, because it is easy to assume otherwise. `getOne` and `getMany` select the whole row including `value`, and `CredentialView` passes that row straight into the form as `initialData` (`credential.tsx:390-393`). The ciphertext is therefore sent over the wire and rendered as the default value of the `value` field — visible in the DOM, in devtools, and in any screenshot, though masked by `type="password"`.

This is **not** a plaintext leak — the value is AES-256-GCM ciphertext and useless without `ENCRYPTION_KEY` — but it is a needless widening of exposure, and the list view serialises the same column for every row on every page load. Two cheap improvements: omit `value` from list responses via a `select`, and on edit, load the credential for its metadata but force the user to re-enter the key (the server then keeps the existing ciphertext when the field is blank).

7. Findings

#	FINDING	SEVERITY	DETAIL AND SUGGESTED FIX
1	`Node.credentialId` column is dead	Medium	0 of 46 rows populated; the real link is `data->>'credentialId'`. Two mechanisms for one relationship. Drop lines 141-142, or populate the column and remove it from `data`.
2	No referential integrity on the credential reference	Medium	Deleting a credential leaves node JSON pointing at nothing; the failure surfaces mid-execution rather than at delete time. Either enforce the real foreign key, or on `remove`, strip `credentialId` from every affected node's `data`.
3	Ciphertext returned to the client by `getOne` / `getMany`	Low	Not a plaintext leak, but unnecessary. Add an explicit `select` that omits `value` from list and metadata queries.
4	Gate asymmetry: `create` premium, `update` / `remove` not	Low	Probably intentional (a lapsed subscriber should still be able to revoke), but undocumented in code. If not intended, align the gates.
5	No `ENCRYPTION_KEY` rotation path	Medium	Salt is stored with the ciphertext, so a single lost key destroys every credential irrecoverably. Add a re-encrypt script and document the recovery procedure before it is needed.
6	PBKDF2 runs synchronously per call	Low	100,000 SHA-512 iterations via `pbkdf2Sync` block the event loop on every encrypt/decrypt. Switch to the `CryptrAsync` export if credential volume grows.
7	Duplicate keys undetectable by design	Informational	Random salt + IV means the same key never produces the same ciphertext (confirmed: 0 duplicates). Fine for GCM, but means no uniqueness check without decrypting.
8	Unrelated: OpenAI executor publishes on the Anthropic channel	Low	`openai/executor.ts:52` calls `anthropicChannel().status(…)` for a missing-credential error, so the OpenAI node's error status is delivered to the wrong subscriber and the UI shows nothing. Line 87 has the mirror problem — it builds `openAiChannel().status(…)` but never passes it to `publish()`. Known copy-paste bug; noted here only because it sits in the credential-error path this document describes.

8. Reference

LOCATION	ROLE
`prisma/schema.prisma:79-99`	`CredentialType` enum and `Credential` model.
`prisma/schema.prisma:141-142`	The unused `Node.credentialId` column and relation.
`src/lib/encryption.ts`	Lazy Cryptr singleton; the only two exports are `encrypt` and `decrypt`.
`src/features/credentials/server/routers.ts`	Six procedures. `create` is the only premium one.
`src/features/credentials/components/credential.tsx`	`CredentialForm` and `CredentialView`. Google Sheets JSON branch at 207-227.
`src/features/credentials/hooks/use-credentials.ts`	Six React Query hooks; toast + cache invalidation on every mutation.
`src/features/executions/components/*/executor.ts`	Six executors, each doing the owner-scoped lookup and inline decrypt.
`src/features/workflows/server/routers.ts:152-163`	Node upsert — the write path that omits `credentialId`.
`node_modules/cryptr/index.js`	Algorithm, parameters, and the `salt | iv | tag | ciphertext` concatenation.
`.env — ENCRYPTION_KEY`	The only thing standing between a database dump and every third-party account.

Method note

Row counts, ciphertext lengths, plaintext-equivalent sizes, the `credentialId` column audit, the seven JSON linkage paths, the duplicate-ciphertext check, and the secret-shaped-string scan were all read directly from the project's Neon database. The encryption parameters and byte layout were read from the installed `cryptr` package. The six-scope executor check was done by inspecting each lookup individually. No credential was decrypted and no plaintext secret appears in this document.

— end of document —